

- 1) Aim: To understand tokens in C++ such as keywords, identifiers, and constants.
- 2) Aim: To learn different types of constants such as integer, real, character, string and backslash constants.
- 3) Aim: To study the features of C++ and its basic structures.
- 4) Aim: To write and execute a simple C++ program without using classes.
- 5) Aim: To understand the process of compiling and running C++ program.

Input:

```
#include <iostream>
using namespace std;
int main() {
    const int rollnumber = 26;
    const float pi = 3.14;
    const char grade = 'A';
    const string name = "Rehan";
    int age = 20;
    float marks = 95.5;
    cout << "Student Name: " << name << endl;
    cout << "Roll Number: " << rollnumber << endl;
    cout << "Grade: " << grade << endl;
    cout << "Age: " << age << endl;
    cout << "Marks: " << marks << endl;
    cout << "Value: " << pi << endl;
    return 0;
}
```

Output

```
Student Name: Sameer
Roll Number: 45
Grade: A
Age: 20
Marks: 85.5
Value: 3.14
```

=== Code Execution Successful ===

6) Aim: Write a program to demonstrate implicit and explicit type casting.

Input:

```
#include <iostream>
using namespace std;
int main() {
int a = 10;
float b = a;
cout << "Implicit Casting (int ? float)" << endl;
cout << "Value of a(int): " << a << endl;
cout << "Value of b(float): " << b << endl;
float x = 9.87f;
int y = int(x);
cout << "\nImplicit Casting (int ? float): " << endl;
cout << "Value of x(float): " << x << endl;
cout << "Value of y(int): " << y << endl;
return 0;
}
```

Output

```
Implicit Casting (int ? float)
Value of a(int): 10
Value of b(float): 10

Implicit Casting (int ? float):
Value of x(float): 9.87
Value of y(int): 9
```

=== Code Execution Successful ===

7) Aim: Write a program to use cin and cout for input and output operations.

Input:

```
#include <iostream>
using namespace std;
int main() {
    int age;
    float marks;
    string name;
    cout << "Enter your name: " << endl;
    cin >> name;
    cout << "Enter your age: " << endl;
    cin >> age;
    cout << "Enter your marks: " << endl;
    cin >> marks;
```

```
    cout << "\n-----output---- " << endl;
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
    cout << "Marks: " << marks << endl;
    return 0;
}
```

Output

```
Enter your name:
Sameer
Enter your age:
8
Enter your marks:
99

-----output----
Name: Sameer
Age: 8
Marks: 99
```

```
=== Code Execution Successful ===
```

8)Aim: To write a program to demonstrate different operations in C++.

Input:

```
#include <iostream>
using namespace std;
int main() {
int a = 10,b = 5, c = 2;
int result1 = a + b + c;
int result2 = (a + b) * c;
int result3 = a / b * c;
int result4 = a + b - c * 3;
cout << "-----Operator Precedence Demonstrate---- " << endl;
cout << "a + b + c " << result1 << endl;
cout << "(a + b) * c " << result2 << endl;
cout << "a / b * c " << result3 << endl;
cout << "a + b - c * 3 " << result4 << endl;
return 0;
}
```

Output

Clear

-----Operator Precedence Demonstrate----

a + b + c 17

(a + b) * c 30

a / b * c 4

a + b - c * 3 9

=== Code Execution Successful ===

9) Aim : To write a program to demonstrate different function types in C++.

←External→

Input:

```
#include <iostream>
```

```
using namespace std;
```

```
extern void show(){
```

```
cout <<"External Function";
```

```
}
```

```
int main() {
```

```
show();
```

```
return 0;
```

```
}
```

Output

External Function

=== Code Execution Successful ===

←Internal→

Input:

```
#include <iostream>
```

```
using namespace std;
```

```
void show(){
```

```
cout <<"Internal Function";
```

```
}
```

```
int main() {
```

```
show();
```

```
return 0;
```

```
}
```

Output

Internal Function

=== Code Execution Successful ===

←Inline→

Input:

```
#include <iostream>
using namespace std;
inline void show(){
    cout <<"External Function";
}
int main() {
    show();
    return 0;
}
```

Output

External Function

=== Code Execution Successful ===

10) Aim: To write a program in C++ to demonstrate the class and object.

Input:

```
#include <iostream>
using namespace std;
class Student{
public:
int a = 101;
void show(){
cout <<"\ sameer";
}
};
int main() {
Student s;
cout <<5.7;
s.show();
}
```

Output

5.7 Sameer

=== Code Execution Successful ===

11) Aim: To write a program in C++ to demonstrate the concept of pass by value and pass by function.

Input:

```
#include <iostream>
using namespace std;
class Student {
public:
void show(int &a) {
a = a + 10;
cout << a << endl;
}
};
int main() {
Student s;
int x = 5;
s.show(x);
cout << x;
}
```

Output

15

15

=== Code Execution Successful ===

12) Aim: To study and implement different types of construction and destructors in C++.

Input:

```
#include <iostream>
using namespace std;
class A {
public:
A(){
cout<<"Default Constructor"<<endl;
}
A(int x){
cout<<"Parameter Constructor"<<endl;
}
A(A &obj){
cout<<"Copy Construcor"<<endl;
}
~A(){
cout<<"Distructor"<<endl;
}
};
int main() {
A a;
A b(2);
A c(b);
return 0;
}
```

Output

```
Default Constructor
Parameter Constructor
Copy Construcor
Distructor
Distructor
Distructor
```

=== Code Execution Successful ===

13) Aim: To study and implement various concept of working with object including accessor and mutator function, static data member , static functions, access modifiers and array of objects.

Input:

```
#include <iostream>
using namespace std;
class A{
private:
string name;
static int roll;
public:
void setName(string x){
name = x;
}
string getName(){
return name;
}
static int getRoll(){
return roll;
}
};
int A::roll=101;
int main() {
A a[2];
a[0].setName("sameer ");
a[1].setName("sameer ");
cout<<a[0].getName();
cout<<a[1].getName();
return 0;
}
```

Output

sameer sameer

=== Code Execution Successful ===

14) Aim: To study and implement the concept of polymorphism.

←Function Overloading→

Input:

```
#include <iostream>
using namespace std;
class A{
public:
void add(int x ,int y){
int z = x+y;
cout<<"\n"<<z;
}
void add(double x ,double y){
double z = x+y;
cout<<"\n"<<z;
}
};
int main() {
A a;
a.add(1 ,4);
a.add(1.2,3.79);
return 0;
}
```

Output

5

4.99

=== Code Execution Successful ===

←Unary Overloading→

Input:

```
#include <iostream>
using namespace std;
class Demo{
public:
int a;
void operator++(){
a = a+1;
}
void show(){
cout<<a;
}
};
int main(){
Demo d;
d.a=5;
++d;
d.show();
return 0;
}
```

Output

6

=== Code Execution Successful ===

←Binary Overloading→

Input:

```
#include <iostream>
using namespace std;
class Demo{
public:
int a;
void getData(){
cout<<"Enter a number: ";
cin>>a;
}
Demo operator+(Demo bb){
Demo cc;
cc.a = a+bb.a;
return cc;
}
void putData(){
cout<<"\n value = "<<a;
}
};
int main(){
Demo aa, bb, cc;
aa.getData();
bb.getData();
cc=aa+bb;
aa.putData();
bb.putData();
cc.putData();
return 0;
}
```

Output

```
Enter a number: 5
Enter a number: 5
```

```
value = 5
value = 5
value = 10
```

=== Code Execution Successful ===

15) Aim: To study and implement the concept of inheritance.

←Single Inheritance→

Input:

```
#include <iostream>
using namespace std;
class person{
public:
void ShowPerson(){
cout<<"show person";
}
};
class Student:public person{
public:
void ShowStudent(){
cout<<"show student";
}
};
int main(){
Student s;
s.ShowStudent();
return 0;
}
```

Output

show student

=== Code Execution Successful ===

←Multilevel Inheritance→

Input:

```
#include <iostream>
using namespace std;
class Person {
public:
void ShowPerson() {
cout << "show person" << endl;
}
};
class Student : public Person {
public:
void ShowStudent() {
cout << "show Student" << endl;
}
};
class Library : public Student {
public:
void ShowLibrary() {
cout << "show library" << endl;
}
};
int main() {
Library l;
Student s;
Person p;
p.ShowPerson();
s.ShowStudent();
l.ShowLibrary();
return 0;
}
```

Output

```
show person
show Student
show library
```

=== Code Execution Successful ===